MODELADO SEMÁNTICO DE POWER BI ASISTIDO POR IA CON GITHUB COPILOT Y MCP

Temas 7 y 8

Proyectos PBIP, trabajo en equipo y gobierno



Tema 7: el modelo y el informe como texto, con GitHub · Tema 8: el agente en el informe, cambios masivos y gobierno

Agenda del día

1. **Tema 7 · Del .pbix al proyecto PBIP**
   Modelo en TMDL, informe en PBIR

2. **Tema 7 · Pausa: Git y GitHub**
   Seis conceptos y el flujo del equipo

3. **Tema 8 · El agente en el informe**
   Crear y cambiar visuales con método

4. **Tema 8 · Cambios masivos y gobierno**
   Documentar, proteger y revisar

5. **Laboratorio final**
   El informe del equipo, de la rama al merge



Al terminar el día

- Explicar **commit, rama y pull request** con sus palabras
- Pedir al agente **un visual nuevo** y validarlo
- Entregar un cambio por **pull request** y revisar el de otro
- Proponer **controles** para usar el agente en producción



TEMA 7

Proyectos PBIP y trabajo en equipo

El modelo y el informe como texto, y un flujo para que varias personas trabajen sobre el mismo proyecto

Del archivo .pbix al proyecto PBIP

Archivo .pbix

- Un solo archivo **binario**: no se puede leer por dentro
- Dos versiones solo se comparan **abriéndolas**
- Un cambio pequeño reemplaza **todo el archivo**
- Trabajar en paralelo termina en **copias con nombres**

Proyecto PBIP

- Una **carpeta** con archivos de **texto**
- Se ve **qué línea cambió**, en qué archivo
- Varias personas trabajan en **archivos distintos**
- Lo leen **Git, VS Code y el agente**

Así se ve un proyecto PBIP

Estructura de carpetas

```
MiProyecto\
├─ MiProyecto.pbip              ← se abre con Desktop
├─ .gitignore                   ← lo que Git no guarda
├─ MiProyecto.SemanticModel\
│  └─ definition\
│     ├─ model.tmdl
│     ├─ relationships.tmdl
│     └─ tables\  (un .tmdl por tabla)
└─ MiProyecto.Report\
   ├─ definition.pbir           ← une el informe con su modelo
   └─ definition\
      └─ pages\
         ├─ pages.json          ← orden de las páginas
         └─ <página>\
            ├─ page.json
            └─ visuals\<visual>\visual.json
```

- **SemanticModel**: el modelo, en **TMDL**
- **Report**: el informe, en **PBIR**
- Cada **página** es una carpeta; cada **visual**, un visual.json
- La caché de datos queda **fuera** de Git (.gitignore)

Dos piezas, dos formas de cambiarlas

Modelo · TMDL

Tablas, relaciones y medidas. El agente lo cambia con el servidor MCP, con aprobación en cada herramienta.

Informe · PBIR

Páginas, visuales y filtros en JSON. El servidor MCP no lo toca: el agente lo cambia editando archivos.

Desktop

Cambia las dos piezas al guardar. No ve en vivo los archivos que otro editó: hay que reabrir el proyecto.

En equipo

Separar las piezas permite que una persona cuide el modelo y otras construyan páginas, con menos choques.



TEMA 7 · PAUSA

Git y GitHub sin complicaciones

Seis conceptos, todos desde la página web de GitHub. No hace falta instalar nada.

Los seis conceptos que vamos a usar

Concepto	Qué es	Piénsenlo como…
Repositorio	La carpeta del proyecto en GitHub, con toda su historia de cambios	El archivador compartido del equipo
Commit	Un cambio guardado, con autor, fecha y mensaje	Una foto con nota al pie: «agregué mi página»
Rama (branch)	Una línea de trabajo aparte; main es la oficial	Una copia de borrador donde nadie más escribe
Pull request	La solicitud para llevar los cambios de una rama a main	Pedir que revisen y aprueben su borrador
Merge	Integrar la rama en main cuando el pull request se aprueba	Pasar el borrador al documento oficial
Conflicto	Dos ramas cambiaron la misma parte de un archivo	Dos personas corrigiendo el mismo párrafo

Git es la herramienta de control de versiones; GitHub es la plataforma web donde se guardan los repositorios y se revisan los cambios.

El flujo del equipo

Nadie escribe directo en main: todo cambio entra por un pull request revisado



Revisar el diff antes de aceptar



Solo lo esperado

En Files changed, ¿aparecen solo los archivos del cambio? Un archivo de más es una pregunta para el autor.



Lectura de líneas

Verde lo agregado, rojo lo quitado. En un visual.json se leen el tipo de visual, los campos y el título.



Lo que el diff no muestra

Si el visual abre, si los números son correctos o si algo se rompió en otro lado. Eso se valida en Desktop.



Comentar y aprobar

Comentarios concretos en la línea, y aprobar solo lo que revisó.

Cómo evitar conflictos en equipo



«Una persona, una rama, una carpeta. Cambios pequeños, revisados pronto.»

- Repartir el trabajo por **archivos o carpetas**: cada página en su carpeta
- Archivos **compartidos** (como pages.json) solo los toca **una persona**
- Pull requests **pequeños** y ramas de **vida corta**
- Si aparece un conflicto, se resuelve **leyendo las dos versiones**, nunca a ciegas

DEMOSTRACIÓN

Una página nueva, de la rama al merge

El recorrido completo en la web de GitHub con el repositorio del equipo

1. Recorrer el repositorio: carpetas, ENCARGOS.md y prompts del equipo
2. Crear la rama pagina-instructor
3. Subir la carpeta de una página con Upload files
4. Abrir el pull request con la plantilla
5. Revisar Files changed y hacer el merge

Repositorio del grupo
en Cursos-CM



TEMA 8 · El agente en el informe, operaciones masivas y gobierno

El agente en el informe y gobierno

Crear y cambiar visuales con el agente, cambiar mucho sin romper nada, documentar y proteger

Dos caminos para que el agente cambie algo

Modelo · servidor MCP

- Herramientas **específicas**: medidas, relaciones, tablas
- **Aprobación** en cada herramienta: ✓ lee, ⚠ escribe
- El servidor **valida** lo que recibe
- Desktop lo ve **en vivo**

Informe · edición de archivos

- Herramienta **edit** sobre los JSON de PBIR
- Revisión al final: **Keep** o **Undo**
- **Nadie valida** el JSON: lo valida Desktop al abrir
- Desktop **no lo ve en vivo**: cerrar sin guardar y reabrir

Crear o cambiar un visual con el agente

El mismo método de siempre, aplicado a los archivos del informe



Cuidados al dejar que el agente edite archivos



La llave, solo cuando se necesita

Activar edit para la tarea y apagarlo al terminar. execute y browser, apagados: en las pruebas el agente quiso ejecutar código para validar.



Un alcance explícito

«Solo la carpeta de mi página.» Sin tocar el modelo ni archivos compartidos como pages.json.



Una salida segura

Keep o Undo al final, y el repositorio como red: lo que no está en un commit se puede descartar.



Revisión humana

El diff del pull request muestra exactamente qué archivos y qué líneas cambió el agente.

Operaciones masivas: cambiar mucho, con método

Operación en lote	Riesgo principal	Cómo se valida
Formatos y carpetas de medidas	Bajo: no cambia ningún número	Revisar el panel de datos
Descripciones de tablas y medidas	Bajo: texto que puede quedar impreciso	Leer una muestra con el dueño del negocio
Ocultar claves y columnas técnicas	Medio: un visual que las usaba	Buscar los nombres en los visual.json
Renombrar tablas o columnas	Alto: medidas, visuales e informes externos	Plan con referencias, consulta DAX y Desktop
Eliminar objetos	Alto: dependencias ocultas	Dependencias primero; respaldo o rama

Regla práctica: lotes pequeños, del menor al mayor riesgo, con una validación al final de cada uno.

Documentar con el agente

Descripciones en el modelo

Una frase de negocio por tabla, columna y medida. Aparecen al pasar el mouse en Desktop y en Copilot de Power BI.

Diccionario de datos

Tablas, columnas, medidas con su DAX, formatos y relaciones, generado desde el modelo real.

Informe de revisión

Qué se cambió, por qué, cómo se validó y qué queda pendiente. Se entrega con el trabajo.

Mensajes de commit y PR

La documentación más honesta: escrita en el momento del cambio, junto al cambio.

Seguridad: qué viaja y quién decide

Qué viaja y hacia dónde

- El agente envía al modelo de IA **el prompt y lo que leen las herramientas**
- Eso incluye **metadatos** y los **resultados de consultas** DAX
- El servidor MCP corre **en su equipo**; el modelo de IA, **en la nube**
- Nunca credenciales ni datos personales en prompts o archivos

Quién decide y con qué controles

- La **organización**: políticas de Copilot, modelos y servidores MCP permitidos
- **VS Code**: Configure Tools, aprobaciones y servidores de solo lectura
- **GitHub**: permisos del repositorio y rama main protegida
- **Power BI**: roles de seguridad por filas (**RLS**), que el agente puede crear con plan y APROBADO

Gobierno: recomendaciones para modelos de producción



Todo en un repositorio

Proyecto PBIP en GitHub o Azure DevOps, main protegida y cambios solo por pull request.



Revisión por otra persona

Quien pide el cambio al agente no es quien lo aprueba en el pull request.



Valores esperados

Un conjunto de consultas de control que se corre antes y después de cada cambio.



Reglas del equipo versionadas

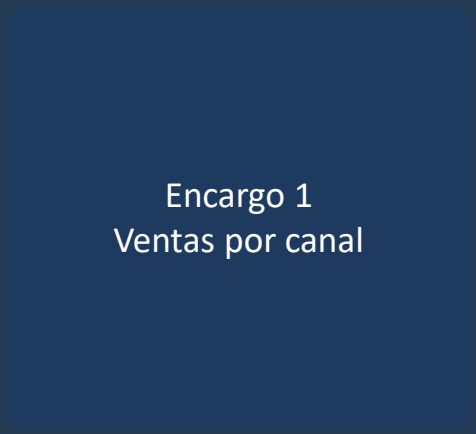
copilot-instructions.md y los prompts del equipo, dentro del repositorio y revisados como el código.

DEMOSTRACIÓN

El agente crea y cambia un visual

Con los prompts del equipo: consultar, planear, aprobar, reabrir Desktop y validar

1. /consultar-encargo: los números antes de construir
2. Activar edit solo para este paso
3. /crear-pagina: plan, revisión y APROBADO
4. Cerrar Desktop sin guardar, reabrir y validar
5. /cambiar-visual y devolver la llave



LABORATORIO

Laboratorio final: el informe del equipo

Como en un proyecto real: primero el modelo, después el informe y al final Git

1. Arranque y su encargo con el agente (solo lectura)
2. Modelo: documentar las 18 medidas en un lote
3. Modelo: rol RLS «Región Caribe» y Ver como
4. Informe: su página con /crear-pagina y un cambio de visual
5. Git: rama, carpeta de su página y pull request
6. Revisión cruzada e integración en main

Un repositorio por grupo
12 encargos

Ideas clave del día y del curso

- PBIP convierte el modelo y el informe en texto que se puede comparar y revisar
- Rama, pull request y merge: nadie escribe directo en main
- El agente cambia el modelo con el servidor MCP y el informe editando archivos: dos niveles de cuidado
- Plantilla, plan, APROBADO, reabrir y validar: también para los visuales
- El agente propone, ustedes aprueban, y el repositorio guarda la historia

Gracias por estos cuatro días. Sigan aprendiendo en Microsoft Learn: Power BI, PBIP y GitHub Copilot.